

Checklist:

Role-Model Staff and Principal Engineers

1. Understand the role and expectations

- ☐ Clarify what “staff,” “principal,” or equivalent titles mean in your company.
- ☐ Identify the full career ladder above senior engineer.
- ☐ Understand the expected scope, blast radius, and influence for your level.
- ☐ Confirm whether your role is expected to operate at team, group, org, or company level.
- ☐ Understand how your role pairs with engineering managers and product managers.
- ☐ Treat staff+ work as a partnership with EMs, PMs, business stakeholders, and senior engineers.
- ☐ Balance your time across building software, strategy/alignment, collaboration, mentoring, and business understanding.
- ☐ Keep enough hands-on connection to the work to make practical decisions.

2. Understand the business

North Stars, KPIs, and OKRs

- ☐ Know your product or team’s North Star.
- ☐ Know the KPIs used to measure progress.
- ☐ Know which company or org OKRs your team supports.
- ☐ Understand how your team’s work maps to revenue, growth, cost, reliability, retention, or customer satisfaction.
- ☐ Question whether the right things are being measured.
- ☐ Look for ways metrics can be gamed.

- ☐ Add countermetrics where a target could create unintended harm.
- ☐ Help translate business goals into engineering goals the team understands.

Product and customers

- ☐ Understand who the customers are.
- ☐ Understand what problems your product solves for them.
- ☐ Use the product yourself where possible.
- ☐ Talk directly with customers, customer support, sales, or user research.
- ☐ Read reviews, support tickets, customer feedback, and competitor material.
- ☐ Understand why customers choose your product.
- ☐ Understand why customers churn, complain, or stop using it.
- ☐ Know your competitors and how they compare.
- ☐ Compare user flows, capabilities, strategy, and customer feedback against competitors.
- ☐ Understand whether your product is a profit center, cost center, platform, internal tool, or strategic enabler.
- ☐ Create or maintain a SWOT analysis for your product.

Company and stakeholders

- ☐ Understand how the company makes money.
- ☐ Understand the business model and unit economics.
- ☐ Know how engineering work connects to revenue, cost, margin, risk, or growth.
- ☐ Have 1:1s with product people.
- ☐ Have 1:1s with business stakeholders.
- ☐ Talk with people outside engineering and product: support, sales, marketing, legal, finance, operations, PR, data, security, design.
- ☐ Pay attention to leadership communication and infer the real priorities.
- ☐ Ask your manager and skip-level leaders about unclear business areas.
- ☐ Read PRDs or equivalent planning documents.
- ☐ Join strategy discussions to listen and learn.
- ☐ Work on cross-functional projects to broaden your business context.
- ☐ Create opportunities for informal conversations with people outside your immediate team.

Public companies, startups, and industry

- ☐ For public companies, read quarterly reports, earnings calls, and analyst questions.

- ☐ Track revenue trends, profitability, investments, risks, and forward-looking commitments.
- ☐ For startups, understand runway, funding, investor priorities, growth areas, and business risks.
- ☐ Ask founders or executives about company priorities when appropriate.
- ☐ Understand your industry's key players, products, trends, and disruptions.
- ☐ Follow relevant industry news and trade publications.
- ☐ Track competitor launches and customer-impacting changes.

3. Collaborate and influence

Internal politics and trust

- ☐ Avoid behavior that appears self-serving, credit-grabbing, inflexible, or manipulative.
- ☐ Be aware that perception matters, even when your intent is good.
- ☐ Avoid bulldozing decisions through authority.
- ☐ Avoid being “all talk, no action” or detached from delivery.
- ☐ Give feedback about problematic political behavior carefully and at the right level.
- ☐ Build trust capital through credibility, reliability, authenticity, and low self-interest.
- ☐ Earn influence by helping others succeed, not by controlling decisions.

Influence others constructively

- ☐ Use influence to get good proposals accepted.
- ☐ Push back on damaging initiatives when needed.
- ☐ Support teammates' good proposals.
- ☐ Help important cross-team work move forward.
- ☐ Ask questions and actively listen before proposing solutions.
- ☐ Explain your point of view clearly.
- ☐ Start proposals with the problem, preferred solution, known unknowns, and tradeoffs.
- ☐ Take sides in design discussions when useful, but explain your reasoning.
- ☐ Choose the best solution, even when it is not your own.
- ☐ Roll up your sleeves and help execute.

Make work visible

- ☐ Share what you are doing with your manager, teammates, and stakeholders.
- ☐ Maintain a work log or regular written updates.
- ☐ Communicate the business impact, risks, challenges, and lessons learned.
- ☐ Give concise weekly or periodic updates when useful.
- ☐ Become a better writer, especially for distributed or large organizations.
- ☐ Use documents, RFCs, ADRs, runbooks, and design notes to scale communication.

Managers and staff+ peers

- ☐ Build a strong partnership with your own manager.
- ☐ Clarify responsibilities with EMs so you do not step on each other's toes.
- ☐ Support EMs in alignment, execution, and technical direction.
- ☐ Partner with managers without publicly undermining them.
- ☐ Get to know staff+ peers face-to-face or through regular discussions.
- ☐ Join or create a staff+ community.
- ☐ Understand your staff+ archetype: tech lead, architect, solver, right hand, or a mix.
- ☐ Learn from engineers with complementary strengths.

Network and help others

- ☐ Find mentors, advisors, and allies in the organization.
- ☐ Work on cross-team projects to build durable relationships.
- ☐ Attend internal trainings and company events.
- ☐ Meet people outside your immediate team.
- ☐ Help colleagues without expecting immediate return.
- ☐ Mentor less experienced engineers.
- ☐ Sponsor engineers by creating visibility, lending support, bringing them into rooms, and advocating for them.
- ☐ Speak up for people who are being overlooked.
- ☐ Help others grow through feedback, pairing, reviews, and coaching.

4. Improve software engineering execution

Coding as a staff+ engineer

- ☐ Keep coding, but accept that you will code less than before.
- ☐ Code in focused bursts when hands-on work matters most.
- ☐ Use coding work to mentor, coach, and lead by example.
- ☐ Pair with engineers, especially when it helps them grow.
- ☐ Jump into struggling projects when your help can unblock delivery.
- ☐ Adapt your style to the team instead of reshaping everything around your preferences.
- ☐ Choose coding work with high impact.
- ☐ Build frameworks, tests, tooling, or infrastructure that others can reuse.
- ☐ Get involved early in projects where architecture and quality decisions matter.
- ☐ Protect time for deep technical work.

Engineering processes

- ☐ Help teams define “done.”
- ☐ Make quality criteria explicit.
- ☐ Establish coding style guidelines where needed.
- ☐ Automate enforcement through linters, CI, or review tooling.
- ☐ Improve code review quality and timeliness.
- ☐ Use post-commit reviews where they fit high-trust teams.
- ☐ Invest in automated testing and testing in production where appropriate.
- ☐ Create scaffolding for new services and components.
- ☐ Improve rollout and experiment hygiene.
- ☐ Remove stale feature flags and stale experiments.
- ☐ Create system health dashboards for owned systems.

CI/CD and delivery

- ☐ Ensure CI gives fast feedback on pull requests.
- ☐ Reduce slow build and test cycles.
- ☐ Modularize code and cache unchanged artifacts.
- ☐ Split tests across machines where helpful.
- ☐ Run smaller test suites first, then full suites.
- ☐ Use CD where safe and appropriate.
- ☐ Prefer automated staged rollouts for risky backend systems.
- ☐ Prefer automated rollback based on health metrics where possible.
- ☐ Understand CI/CD setup, maintenance cost, and developer experience.

- ☐ Watch for CI/CD systems becoming slow as codebases grow.

Trunk-based development and feature flags

- ☐ Understand whether trunk-based development fits your context.
- ☐ Prefer a single source of truth where practical.
- ☐ Encourage frequent commits and integration.
- ☐ Keep trunk healthy through continuous integration.
- ☐ Use feature flags to decouple deploy from release.
- ☐ Use feature flags for staged rollouts, native/mobile constraints, experiments, and incremental delivery.
- ☐ Track and remove obsolete feature flags.
- ☐ Avoid feature-flag debt.

Codebase structure

- ☐ Understand the tradeoffs between monorepos and multiple repositories.
- ☐ Consider monorepos when clarity of dependencies, refactoring, and integration testing matter.
- ☐ Watch for monorepo scale problems: checkout time, tooling, build performance.
- ☐ Understand tradeoffs between monoliths and microservices.
- ☐ Avoid blindly splitting systems into services.
- ☐ Avoid letting monoliths become too large to change safely.
- ☐ Look for modular structures that make pragmatic middle ground possible.
- ☐ Organize microservices into logical domains where service count grows.

Developer productivity tools

- ☐ Check whether your company has a service catalog.
- ☐ Make ownership, on-call rotation, onboarding, and service dependencies discoverable.
- ☐ Improve code search across the whole codebase.
- ☐ Ensure code search supports regular expressions, cross-references, definitions, and speed.
- ☐ Consider developer portals for catalogs, templates, docs, APIs, and plugins.
- ☐ Identify developer productivity bottlenecks before introducing new tools.
- ☐ Evaluate cloud development environments where local setup is slow or inconsistent.
- ☐ Weigh CDE benefits against setup cost, customization gaps, and vendor lock-in.
- ☐ Evaluate AI coding assistants and tools carefully.

- ☐ Consider AI use cases such as reviews, tests, refactoring, compliance checks, and stale flag removal.
- ☐ Think carefully about data ownership, retention, and source code exposure when using AI tools.
- ☐ Decide whether to build, buy, or adopt tools based on scale, capability, cost, and strategic importance.

Compliance, privacy, and security

- ☐ Know the regulations relevant to your product or domain.
- ☐ Protect PII and limit access to people who truly need it.
- ☐ Understand GDPR, PCI DSS, HIPAA, FERPA, FCRA, accessibility, or other relevant requirements.
- ☐ Avoid logging PII.
- ☐ Anonymize or redact sensitive data in logs.
- ☐ Provide guidance on where PII may and may not be stored.
- ☐ Audit logs, exports, screenshots, support tools, and tickets for sensitive data exposure.
- ☐ Prepare seriously for compliance audits.
- ☐ Partner with legal, privacy, compliance, and security teams.
- ☐ Follow secure coding practices.
- ☐ Track dependency vulnerabilities.
- ☐ Understand common threat vectors in your domain.
- ☐ Participate in or learn from penetration testing.
- ☐ Involve security engineers early in planning and risky changes.

5. Build reliable software systems

Own reliability

- ☐ Treat reliability as part of your responsibility.
- ☐ Define reliability objectives and measurable indicators.
- ☐ Use OKRs or KPIs to drive improvements in latency, throughput, cost, uptime, or reliability.
- ☐ Partner with engineering managers to prioritize reliability work.
- ☐ Bring data to reliability conversations.
- ☐ Advocate for engineering bandwidth when systems are unreliable.

Logging

- ☐ Define log levels and when to use them.
- ☐ Define log structure, timestamps, time zones, and required fields.
- ☐ Use automated logging where possible.
- ☐ Define log retention policies.
- ☐ Use logging levels consistently across environments.
- ☐ Log useful events: auth decisions, data access, system changes, data changes, invalid input, resource limits, health events, startup/shutdowns, backups, failures.
- ☐ Ensure every event has a timestamp, system, user, action, status, priority, and reason where relevant.
- ☐ Build or adopt a logging framework that makes the right thing easy.
- ☐ Regularly review logged entries and remove any unsafe or unnecessary logs.

Monitoring

- ☐ Monitor uptime.
- ☐ Monitor CPU, memory, disk, and resource usage.
- ☐ Monitor response times, including p50, p95, and p99.
- ☐ Monitor error rates.
- ☐ Monitor backend HTTP status codes and latency.
- ☐ Monitor web performance metrics such as page load time and Core Web Vitals.
- ☐ Monitor mobile startup time, crash rate, and bundle size.
- ☐ Monitor business metrics that indicate whether the system is actually healthy.
- ☐ Track customer onboarding, conversion, usage, revenue, support tickets, retention, and churn where relevant.
- ☐ Watch for situations where infrastructure metrics look healthy but business metrics show failure.

Alerting

- ☐ Define what “healthy” and “unhealthy” mean for each system.
- ☐ Alert on customer-visible or business-critical failures.
- ☐ Alert on regressions that have previously caused outages.
- ☐ Categorize alerts by urgency.
- ☐ Separate urgent alerts from non-urgent alerts.
- ☐ Track alert noise.
- ☐ Measure alert precision and recall.

- ☐ Review whether alerts fired for actual incidents.
- ☐ Review incidents where no alert fired.
- ☐ Use static thresholds where simple and predictable.
- ☐ Use anomaly detection where patterns are complex.
- ☐ Tune anomaly detection to avoid excessive false positives.
- ☐ Combine static thresholds and anomaly detection where useful.

On-call

- ☐ Ensure each production system has clear on-call ownership.
- ☐ Define primary, secondary, and escalation paths.
- ☐ Keep on-call rotations healthy in size and frequency.
- ☐ Watch for on-call burnout.
- ☐ Reduce noisy, non-actionable alerts.
- ☐ Ensure on-call engineers are not overloaded with normal project work during heavy on-call weeks.
- ☐ Define whether on-call work replaces, reduces, or combines with regular work.
- ☐ Create and maintain alert runbooks.
- ☐ Include diagnostic steps, mitigation steps, relevant previous incidents, dashboards, and links in runbooks.
- ☐ Keep runbooks updated after incidents.

Incident management

- ☐ Define how incidents are detected.
- ☐ Define how incidents are declared.
- ☐ Define severity levels and criteria.
- ☐ Assign incident roles clearly.
- ☐ Identify an incident commander or equivalent role.
- ☐ Mitigate quickly before over-focusing on root cause.
- ☐ Roll back or use known mitigations first when appropriate.
- ☐ Communicate with stakeholders during incidents.
- ☐ Verify that mitigation worked.
- ☐ Hold incident reviews or postmortems.
- ☐ Focus postmortems on learning, not blame.
- ☐ Track follow-up actions to completion.
- ☐ Improve systems, runbooks, alerts, and processes after incidents.
- ☐ Treat incident management as a learning system.

Resilient systems

- ☐ Define SLIs and uptime targets during planning.
- ☐ Plan for failure modes.
- ☐ Plan expected load and peak load.
- ☐ Plan redundancy and data replication.
- ☐ Plan what to monitor and alert on.
- ☐ Code defensively.
- ☐ Pay attention to error states and error mapping.
- ☐ Manage state carefully.
- ☐ Identify and handle unknown states.
- ☐ Test graceful degradation.
- ☐ Test retries.
- ☐ Test circuit breakers.
- ☐ Test datacenter or region failover.
- ☐ Test disaster recovery and backups.
- ☐ Keep checking system behavior after production launch.

6. Make strong architecture decisions

Simplicity and communication

- ☐ Start with the simplest architecture that could work.
- ☐ Avoid complex jargon when simple language is enough.
- ☐ Know the relevant technical, business, and internal jargon.
- ☐ Use jargon only when it helps shared understanding.
- ☐ Explain the architecture in terms that less-experienced engineers can understand.
- ☐ Use sketches, boxes, arrows, documents, or ADRs to communicate decisions.

Architecture debt

- ☐ Identify architecture debt separately from code-level tech debt.
- ☐ Watch for standalone services created only to move faster.
- ☐ Watch for monoliths becoming too hard to change.
- ☐ Watch for non-functional degradation: performance, scalability, reliability.
- ☐ Watch for dated languages and frameworks becoming risky.

- ☐ Revisit older decisions when context changes.
- ☐ Avoid optimizing only for short-term shipping speed.

One-way and two-way door decisions

- ☐ Classify architecture decisions as reversible or hard to reverse.
- ☐ Treat feature flags, naming, test frameworks, linters, and similar changes as mostly reversible.
- ☐ Treat protocol choices, cloud/on-prem moves, database model choices, native app architecture, and major rewrites as harder to reverse.
- ☐ Recognize that two-way-door decisions can become one-way over time.
- ☐ Prototype risky one-way-door decisions.
- ☐ For reversible decisions, avoid over-analysis.
- ☐ For hard-to-reverse decisions, do deeper homework.
- ☐ Reflect after decisions to improve future judgment.

Blast radius

- ☐ Estimate how many teams, systems, and customers a decision affects.
- ☐ Identify whether the blast radius is team-local, org-wide, company-wide, or customer-facing.
- ☐ Expect more pushback for high-blast-radius decisions.
- ☐ Consider customer impact and churn risk.
- ☐ Shrink blast radius where possible.
- ☐ Use adapters, migrations, parallel paths, or gradual rollouts to reduce impact.
- ☐ Avoid high-blast-radius changes unless the business case is strong.

Scalability

- ☐ Clarify whether scalability means new business use cases, traffic growth, data growth, or operational scale.
- ☐ Design for future product extensions only when business plans justify it.
- ☐ Understand business roadmap before building extensibility.
- ☐ Consider horizontal and vertical scaling.
- ☐ Consider sharding, caching, messaging, replication, and CDNs where appropriate.
- ☐ Avoid building scalable architecture for hypothetical needs the business does not have.
- ☐ Build “good enough” architecture that can evolve.

Architecture and business priorities

- ☐ Align architecture work with business goals.
- ☐ Tie architecture improvements to product delivery, reliability, cost, speed, or customer impact.
- ☐ Avoid architecture work that is disconnected from current business needs.
- ☐ Bundle architecture improvements with business-relevant projects where possible.
- ☐ Balance long-term architecture health with near-term product delivery.
- ☐ Accept that good-enough architecture is often better than perfect architecture.
- ☐ Keep architecture close to where implementation happens.
- ☐ Stay involved in code, reviews, design discussions, and operational reality.
- ☐ Coach others to make better architectural decisions rather than becoming a bottleneck.

Architect traits to watch for

- ☐ Avoid becoming an ivory-tower architect.
- ☐ Avoid being painfully precise at the expense of progress.
- ☐ Avoid relying only on theory without practical context.
- ☐ Avoid endless philosophical debate.
- ☐ Avoid using language or jargon to exclude others.
- ☐ Avoid being a walk-away advisor who does not stay for implementation.
- ☐ Cultivate practical traits: coding machine, integrator, approachable mentor, detailed documenter.
- ☐ Balance new-and-shiny exploration with old-school pragmatism.
- ☐ Pair theoretical and practical engineers where possible.
- ☐ Reflect on which architect archetypes you and your peers resemble.

7. Personal operating model

- ☐ Keep learning about your domain, industry, business, and engineering practices.
- ☐ Find problems worth solving, not just assigned tasks.
- ☐ Take initiative when you see important gaps.
- ☐ Unblock yourself and the teams around you.
- ☐ Improve practices across your group, not just your own work.
- ☐ Mentor senior and less experienced engineers.

- ☐ Build relationships before you need influence.
- ☐ Communicate clearly, especially in writing.
- ☐ Maintain visibility into business impact.
- ☐ Stay close enough to implementation to make grounded decisions.
- ☐ Help the organization execute better.
- ☐ Sponsor and support others.
- ☐ Avoid becoming detached from customers, code, operations, or business reality.
- ☐ Balance strategy, software delivery, collaboration, reliability, architecture, and people growth.